

AGENCYOS

Automation, Events & Job System

Document 23 of the AgencyOS Master Documentation Set

TRIGGERS • EVENTS • QUEUES • WORKERS • SCHEDULED JOBS • RETRIES • IDEMPOTENCY •
AGENT-TO-AGENT EVENTS • ESCALATION

Purpose: Define the event-driven automation backbone that allows AgencyOS to execute reliable workflows across CRM, sales, projects, AI agents, Finance, QA, deployment, communication, Customer Success and maintenance without requiring constant human intervention.

1. Core Automation Principles

- Automation is event-driven wherever practical.
- Critical business state remains authoritative in the domain database.
- Events trigger work; events do not replace transactional state.
- Every asynchronous job has an explicit identity, status and retry policy.
- Jobs must be safe to retry.
- Duplicate events must not create duplicate business outcomes.
- Long-running work must not block synchronous API requests.
- High-risk actions remain subject to permissions and policy gates.
- Agents cannot bypass workflow controls by emitting arbitrary events.

2. Automation Architecture

TRIGGER → EVENT → EVENT BUS/QUEUE → JOB → WORKER → TOOL/SERVICE → RESULT
→ DOMAIN UPDATE → FOLLOW-UP EVENT.

- Scheduled workflows enter the same controlled job system.
- Agent-to-agent work uses typed events/tasks rather than unrestricted direct calls.
- Escalation routes failed or blocked jobs to another agent, human owner or Admin.

3. Trigger Types

- Domain triggers: record created, updated, approved, rejected, completed.
- State triggers: status transition.
- Time triggers: due date, reminder, expiry, renewal window.
- Webhook triggers: external integration event.
- User triggers: Admin/client/agent action.
- Threshold triggers: payment threshold, SLA threshold, health score change.
- System triggers: provider outage, job failure, security alert.
- Recurring triggers: daily/weekly/monthly maintenance jobs.

4. Trigger Rules

- Every trigger declares source, event type, scope and conditions.
- Trigger evaluation is deterministic where possible.
- Sensitive triggers require authorization.
- Trigger execution is recorded.
- Trigger loops are prevented.
- Trigger conditions must not silently mutate critical business state without a policy-controlled action.

5. Event Model

- Every event has event_id.
- event_type.
- aggregate_type and aggregate_id.
- organization_id.
- project_id where applicable.

- actor_id/type.
- occurred_at.
- schema_version.
- correlation_id.
- causation_id.
- payload.
- Events should be immutable.

6. Event Categories

- CRM events.
- Sales/commercial events.
- Onboarding events.
- Project events.
- Scope/change events.
- Design/prototype events.
- Development/build events.
- QA/release events.
- Finance/payment events.
- Communication events.
- AI/agent events.
- Handover/completion events.
- Customer Success/maintenance events.
- Security/governance events.
- System/integration events.

7. Canonical Business Events

- LeadCreated.
- LeadQualified.
- QuoteCreated.
- QuoteAccepted.
- ProjectCreated.
- ScopeApproved.
- ScopeFrozen.
- ChangeRequestSubmitted.
- ChangeRequestApproved.
- DesignApproved.
- PrototypeApproved.
- DevelopmentTaskReady.
- BuildCreated.
- TestRunCompleted.
- ReleaseApproved.
- PaymentSubmitted.
- PaymentVerified.

- MilestoneCompleted.
- DeploymentSucceeded.
- HandoverReady.
- ClientAccepted.
- ProjectCompleted.
- MaintenanceRenewalDue.
- SupportTicketCreated.
- CustomerHealthChanged.
- UpsellOpportunityCreated.

8. Event Bus / Queue

- Use durable queues for asynchronous work.
- Support priority where required.
- Support delayed jobs.
- Support dead-letter queues.
- Support retry scheduling.
- Preserve message identity.
- Track queue lag.
- Prevent unauthorized cross-tenant consumption.

9. Queue Types

- Critical workflow queue.
- AI agent queue.
- Communication queue.
- Finance queue.
- Development/build queue.
- QA/testing queue.
- Deployment queue.
- Customer Success queue.
- Scheduled-job queue.
- Low-priority analytics/reporting queue.
- Exact implementation may combine queues where operationally appropriate, but priority isolation must remain possible.

10. Job Model

- Job ID.
- Job type.
- Organization ID.
- Project ID where applicable.
- Trigger event ID.
- Payload/reference.
- Priority.
- Status.

- Attempt count.
- Max attempts.
- Scheduled time.
- Started time.
- Completed time.
- Last error.
- Worker ID.
- Correlation ID.
- Idempotency key.

11. Job States

QUEUED → CLAIMED → RUNNING → SUCCEEDED.

- Failure states: RETRY_WAIT, FAILED, DEAD_LETTERED, CANCELLED.
- Human/agent intervention state: BLOCKED or ESCALATED.
- Jobs must not remain RUNNING forever; lease/timeout recovery is required.

12. Worker Model

- Workers consume only authorized queue types.
- Worker claims a job using a lease/visibility timeout.
- Worker validates job payload.
- Worker checks permissions/policy.
- Worker executes.
- Worker records result.
- Worker emits follow-up event.
- Worker acknowledges job only after durable success state is recorded.

13. Worker Isolation

- Workers are scoped by capability.
- Finance workers do not receive arbitrary development jobs.
- Deployment workers have restricted production permissions.
- AI workers operate with agent/task permissions.
- Workers should not receive raw secrets unless explicitly required through secure mechanisms.

14. Idempotency

- Every retryable job has an idempotency key where business duplication is possible.
- Use unique constraints or idempotency records for critical side effects.
- Webhook events use provider event IDs.
- Payment processing is strictly idempotent.
- Message sending uses provider/message idempotency where supported.
- Project creation from repeated conversion events must not create duplicate projects.
- Agent retries must not create duplicate tasks or approvals.

15. Exactly-Once Reality

- Do not assume distributed systems provide true exactly-once execution.
- Design for at-least-once delivery plus idempotent handlers.
- Atomic state update + outbox/event publication should be used for important domain transitions.
- Side effects are recorded with durable identifiers.

16. Transactional Outbox

- When a database transaction changes authoritative state and must emit an event, write the event to an outbox in the same transaction.
- A dispatcher publishes pending outbox records.
- Published state is tracked.
- Failures can be retried.
- This prevents a successful database update from losing its corresponding event.

17. Event Ordering

- Use aggregate/entity ordering where required.
- Do not assume global ordering across all events.
- Handlers validate current state before applying transitions.
- Out-of-order events can be delayed, ignored or reconciled according to event type.

18. Event Versioning

- Events have schema versions.
- Consumers must handle supported versions.
- Breaking event changes require migration/version strategy.
- Historical events remain interpretable.
- Payload changes must not silently alter business meaning.

19. Scheduled Jobs

- Renewal reminders.
- Payment follow-ups.
- Overdue checks.
- Client follow-ups.
- QA/release checks.
- Maintenance reminders.
- Customer health reviews.
- Backup verification.
- Provider health checks.
- Usage reports.
- Cleanup/retention tasks.
- Daily/weekly/monthly jobs must be timezone-aware.

20. Scheduler Rules

- Scheduled jobs have stable job identifiers.
- Repeated scheduler execution must be idempotent.

- Missed schedules have catch-up policy.
- Timezone is explicit.
- Paused organization/project workflows are respected.
- Disabled integrations do not generate endless retries.

21. Delayed Jobs

- Support execute-at timestamps.
- Use delayed queue/scheduler.
- Validate state at execution time.
- Do not assume conditions remain unchanged between scheduling and execution.
- Example: a renewal reminder should re-check whether renewal already occurred before sending.

22. Retry Policy

- Retry transient errors.
- Do not retry permanent validation or authorization errors blindly.
- Use exponential backoff with jitter.
- Respect provider Retry-After where available.
- Set maximum attempts.
- Set maximum elapsed retry window.
- Persist retry state.
- After exhaustion, dead-letter or escalate.

23. Retry Classification

- TRANSIENT: timeout, temporary provider outage, rate limit.
- PERMANENT: invalid payload, missing required resource, denied permission.
- CONDITIONAL: dependency not ready, waiting for client, payment pending.
- SECURITY: suspicious/unauthorized action—stop and escalate rather than retry.

24. Dead-Letter Queue

- Store failed jobs that cannot complete automatically.
- Preserve original job/event context.
- Store failure reason.
- Store attempts and timestamps.
- Allow authorized retry/replay.
- Allow cancellation.
- Audit manual replay.

25. Escalation System

Agent retry → alternate agent → human owner → PM/Admin escalation according to policy.

- Escalation has severity, reason, owner and deadline.
- Repeated failures increase escalation level.
- Critical financial/security/deployment failures use specialized escalation paths.

26. Escalation States

OPEN → ACKNOWLEDGED → IN_PROGRESS → RESOLVED → CLOSED.

- Can be REOPENED.
- Escalations have SLA/priority where configured.

27. Agent-to-Agent Events

- Agents communicate through typed tasks/events.
- Each event names sender agent, recipient capability/agent, project scope, task ID and correlation ID.
- Recipient validates permissions and current state.
- Agents do not share unrestricted memory or credentials.
- Agent handoff includes required context and completion evidence.

28. Agent Handoff Example

PM Agent → UI Agent: Design task with approved scope version.

UI Agent → PM Agent: Design completion + version + screens + prototype status.

PM Agent → Developer Agent: Approved design + scope + development task.

Developer Agent → QA Agent: Build artifact + commit + test evidence.

QA Agent → Deployment Agent: Approved release candidate + readiness evidence.

Deployment Agent → PM/Customer Success: Production validation result.

29. Agent Event Guardrails

- Recipient cannot accept a task outside its capabilities.
- Sender cannot grant permissions it does not possess.
- Event payload is treated as data, not authorization.
- High-risk actions require policy/approval even if another agent requests them.
- Agent event loops are detected and limited.

30. Automation Conditions

- AND/OR conditions.
- State conditions.
- Time conditions.
- Threshold conditions.
- Role/permission conditions.
- Organization settings.
- Feature flags.
- Provider availability.
- Approval state.
- Payment state.
- Conditions are evaluated immediately before sensitive execution.

31. Automation Workflows

- Lead conversion automation.
- Onboarding automation.
- Scope approval reminders.
- Design review reminders.
- Development task assignment.
- QA trigger after build.
- Invoice/payment notifications.
- Deployment gates.
- Handover checklist.
- Completion notification.
- Customer Success follow-up.
- Renewal reminders.
- Maintenance ticket routing.
- Upsell opportunity creation.

32. Workflow Orchestration

- Complex workflows have explicit workflow IDs.
- Workflow instances track current step.
- Steps have dependencies.
- Workflow state is durable.
- Paused workflows can resume.
- Failed steps can retry or escalate.
- Compensation/recovery actions are defined where necessary.
- Workflow completion is evidence-based.

33. Long-Running Workflows

- Do not hold HTTP requests open for long AI/build/deployment tasks.
- Persist workflow state.
- Use jobs/events.
- Allow progress updates.
- Allow cancellation where safe.
- Allow resumption after worker restart.

34. Cancellation

- Queued jobs can usually be cancelled.
- Running jobs can be cancelled only if the underlying operation supports safe cancellation.
- Critical financial/deployment operations may require controlled completion or rollback.
- Cancellation is audited.

35. Job Priority

- Security/critical production incidents.

- Financial verification.
- Production deployment/recovery.
- Client-blocking project work.
- Normal agent tasks.
- Communication.
- Maintenance.
- Analytics/cleanup.
- Priority is configurable and must not allow untrusted users to starve critical queues.

36. Backpressure

- Detect queue growth.
- Limit concurrent workers.
- Throttle low-priority tasks.
- Prevent agent loops from flooding queues.
- Use provider rate limits.
- Alert on sustained queue lag.

37. Concurrency Control

- Use leases/visibility timeouts.
- Prevent two workers from owning the same job simultaneously.
- Use database locking or optimistic concurrency for shared state.
- Make handlers safe if duplicate execution still occurs.

38. Job Timeouts

- Each job type has a timeout.
- Long tasks use heartbeats/progress where supported.
- Timed-out jobs become retryable or escalated.
- Do not leave abandoned RUNNING jobs indefinitely.

39. Monitoring & Observability

- Queue depth.
- Queue latency.
- Job success/failure rate.
- Retry count.
- Dead-letter count.
- Worker health.
- Execution duration.
- Event processing lag.
- Agent handoff failures.
- Escalation count.
- Scheduled-job misses.
- Provider-related failures.

40. Correlation & Tracing

Carry correlation_id across trigger → event → job → worker → tool → external integration.

- Carry causation_id to identify what caused an event.
- Link agent runs and human actions.
- This enables full workflow tracing without relying on message text.

41. Automation Audit

- Trigger fired.
- Event published.
- Job created.
- Job claimed.
- Job retried.
- Job failed.
- Job dead-lettered.
- Job replayed.
- Escalation created.
- Agent handoff.
- Human intervention.
- Workflow completed/cancelled.

42. Failure Recovery

Worker crash → job lease expires and job is recoverable.

Provider outage → retry/fallback.

Database transient error → retry transaction safely.

Event consumer crash → event remains available.

Partial workflow → resume from durable state.

External side effect uncertain → reconcile before retrying dangerous operation.

43. Reconciliation Jobs

- Payment reconciliation.
- Webhook reconciliation.
- WhatsApp message status reconciliation.
- Deployment status reconciliation.
- Build status reconciliation.
- AI provider health reconciliation.
- Maintenance/renewal state reconciliation.
- Reconciliation jobs repair drift without blindly overwriting authoritative records.

44. Automation Security

- Queue access is permission-scoped.

- Job payloads are tenant-scoped.
- Workers use least privilege.
- Secrets are referenced securely.
- Untrusted event payloads cannot grant permissions.
- Privileged jobs require policy checks.
- Manual replay requires authorization.

45. Automation Cost Controls

- Limit repeated AI retries.
- Cap agent task loops.
- Set maximum job attempts.
- Set per-workflow budgets where needed.
- Track AI/provider usage.
- Prevent runaway scheduled jobs.
- Alert on abnormal execution volume.

46. Automation Configuration in Admin Panel

- Enable/disable workflows.
- Configure schedules.
- Configure retry limits.
- Configure queue priorities.
- Configure escalation rules.
- Configure agent routing.
- Configure provider fallback.
- Configure notification policies.
- View queues/jobs/events.
- Retry/cancel/dead-letter authorized jobs.
- All high-impact changes are audited.

47. Automation Testing

- Unit test trigger conditions.
- Test event schemas.
- Test idempotency.
- Test duplicate delivery.
- Test retries.
- Test backoff.
- Test dead-letter.
- Test worker crash recovery.
- Test scheduled-job duplication.
- Test out-of-order events.
- Test authorization failures.
- Test cross-tenant isolation.
- Test agent loops.

- Test escalation.

48. Definition of Done

- Triggers are explicit and testable.
- Events have stable schemas and versions.
- Queues are durable.
- Workers are scoped.
- Jobs have lifecycle states.
- Retries are bounded.
- Idempotency is enforced.
- Outbox pattern exists for critical event publication where needed.
- Dead-letter handling exists.
- Scheduled jobs are timezone-aware and idempotent.
- Agent-to-agent communication is typed and permission-aware.
- Escalation is automatic for defined failures.
- Long-running work is durable/resumable.
- Monitoring and tracing exist.
- Admin can control automation.
- Automation cannot bypass security, finance, approval or deployment gates.

49. Automation Matrix

Trigger	Event/Job	Primary Worker/Agent	Success	Failure/Escalation
Lead created	LeadCreated	Sales Agent	Qualified/next action	Retry → Sales/Admin
Quote accepted	QuoteAccepted	PM/Onboarding	Project created	Escalate if blocked
Scope approved	ScopeApproved	PM/UI	Design workflow starts	PM review
Build created	BuildCreated	QA Agent	Test run complete	Developer/QA escalation
Tests pass	ReleaseApproved candidate	Deployment	Production validated	Rollback/Admin
Payment webhook	Payment event	Finance	Verified/reconciled	Finance review
Handover ready	HandoverReady	PM/Customer Success	Client acceptance	PM escalation
Renewal due	RenewalDue	Customer Success	Renewed	Retention/Sales
Job failure	JobFailed	Escalation workflow	Recovered/closed	Human/Admin

50. Final Automation Architecture

TRIGGER → DOMAIN EVENT → OUTBOX/EVENT BUS → DURABLE QUEUE →
 WORKER/AGENT → POLICY CHECK → TOOL/INTEGRATION → DURABLE RESULT →
 FOLLOW-UP EVENT → NEXT WORKFLOW

FAILURE PATH: RETRY → BACKOFF → FALLBACK/ALTERNATE WORKER → DEAD LETTER
 → ESCALATION → HUMAN/ADMIN → RECOVERY → REPLAY/RESUME

Key outcome: AgencyOS should operate as a reliable event-driven system where routine work happens automatically, agents can hand work to one another, external failures recover safely, and humans are interrupted only when a decision, exception or policy gate genuinely requires them.